Pure functions and referential transparency

A pure function returns the same output for the same input and has no side effects. A side effect changes something outside the returned value, such as printing to a console, writing a file, or updating shared mutable state. Reading the current clock also prevents a function from depending only on its explicit inputs.

Referential transparency means that an expression can be replaced by its value without changing the program's observable behavior. It makes equational reasoning possible: we can reason about code by substituting equals for equals.

Haskell examples:

```
square x = x * x
square 4 == 16
```

The expression square 4 can be replaced with 16. A function that adds the current clock time to x would not have this property. Pure code is easier to test because a test can supply inputs and compare returned values without arranging external state.

Original educational material / CC BY 4.0 / Bilal Amin

Types, immutability, and explicit effects

A type signature describes the inputs and result of a function. In Haskell, square :: Int -> Int means that square accepts an Int and returns an Int. A list of integers has type [Int]. A function accepting two integers and returning an integer can have type Int -> Int -> Int. Arrows associate to the right.

An immutable value is not updated in place. Binding a new name or constructing a new list does not change the original value. This avoids surprising changes through shared references.

Purity does not mean a program cannot perform input or output. Haskell represents such effects explicitly using IO actions. For example, putStrLn :: String -> IO () describes an action that prints a string. Keeping an effectful shell around a pure core lets the core remain simple to test.

```
add :: Int -> Int -> Int
add x y = x + y
```

A type signature alone is not a proof that a function computes the intended answer. Tests and reasoning are still needed.

Original educational material / CC BY 4.0 / Bilal Amin